📘 Hously Finttech Realty — Technical Documentation

Version 1.0.0 | April 2026 | Confidential

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Project Structure
5. Database Schema
6. API Reference
7. Authentication Flow

---

1. Project Overview

Hously Finttech Realty is a full-stack web platform combining real estate advisory and fintech services. It provides a public-facing marketing site with career/job listings, a contact/project inquiry system, and a protected admin dashboard for internal team management.

Property	Value
Product Name	Hously Finttech Realty
Type	Full-Stack Web Application
Frontend	React 18 + Vite + Tailwind CSS
Backend	Node.js + Express 5
Database	MySQL 8
Auth	JWT (
–	–
httpOnly Cookies) + Refresh Tokens	
Frontend Port	`5173`
Backend Port	`5000`
Deployment Ready	✅ Yes

---

2. Technology Stack

2.1 Frontend

Technology	Version	Purpose
React	18.x	UI Component Library
Vite	Latest	Build Tool & Dev Server
React Router DOM	v6	Client-Side Routing
Axios	Latest	HTTP Client with Interceptors
Swiper.js	Latest	Testimonial Carousel
Tailwind CSS	v3	Utility-First Styling

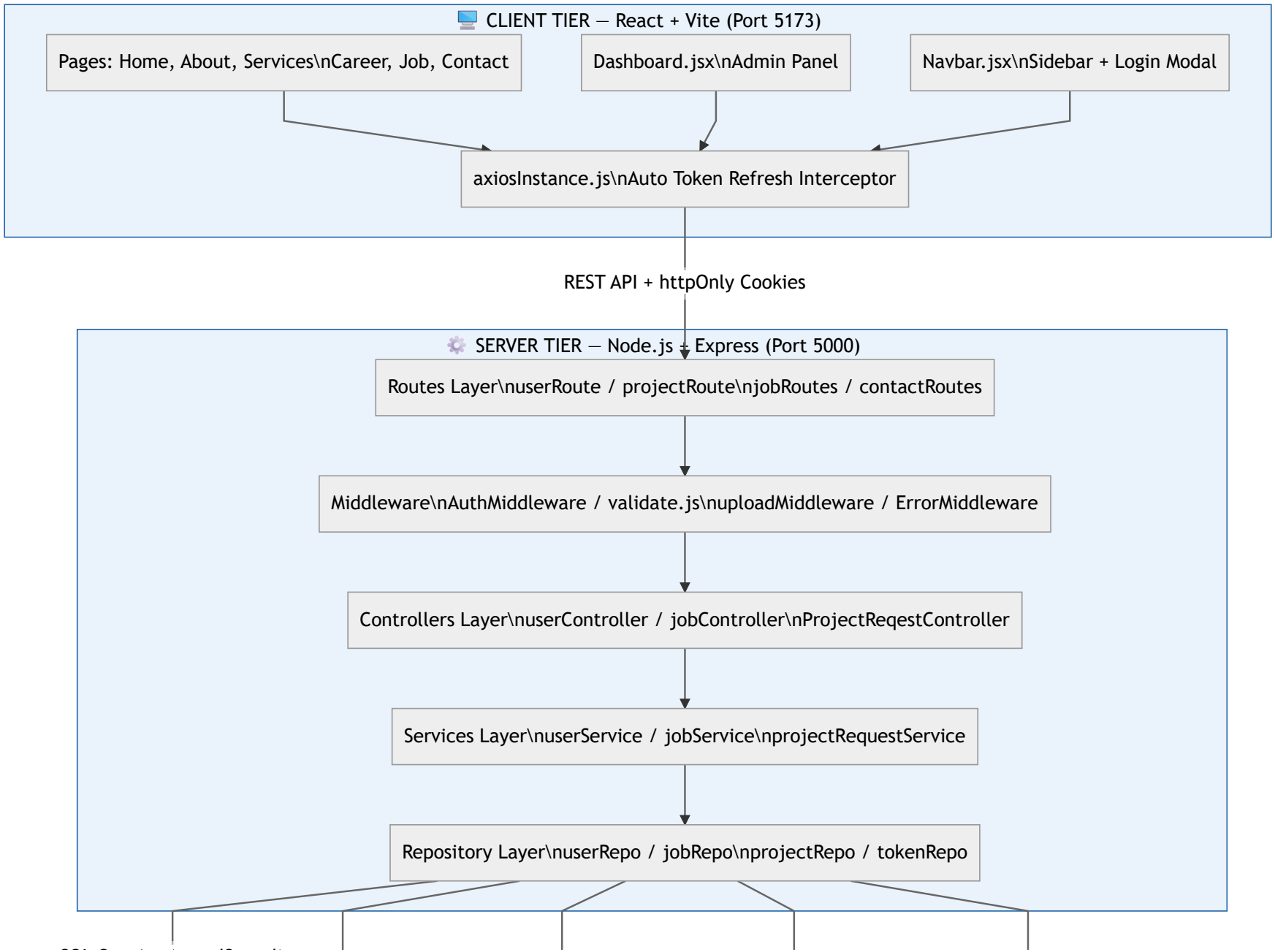
2.2 Backend

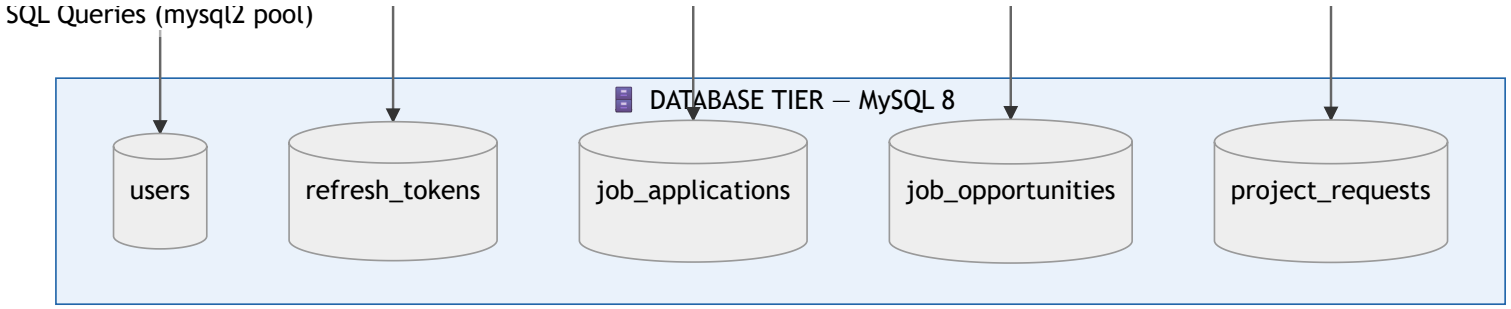
Technology	Version	Purpose
Node.js	18+	JavaScript Runtime
Express	^5.2.1	Web Framework
MySQL2	^3.22.0	MySQL Driver (Promise-based)
bcrypt	^6.0.0	Password Hashing
jsonwebtoken	^9.0.3	JWT Token Generation & Verification
cookie-parser	^1.4.7	Cookie Parsing Middleware

Technology	Version	Purpose
cors	^2.8.6	Cross-Origin Resource Sharing
dotenv	^17.4.2	Environment Variable Management
express-validator	^7.3.2	Input Validation
multer	^2.1.1	File Upload (Resume as BLOB)
nodemon	^3.1.14	Dev Hot-Reload

---

3. System Architecture


CLIENT TIER — React + Vite (Port 5173)
Pages: Home, About, Services\nCareer, Job, Contact
Dashboard.jsx\nAdmin Panel
Navbar.jsx\nSidebar + Login Modal
axiosInstance.js\nAuto Token Refresh Interceptor
REST API + httpOnly Cookies
SERVER TIER — Node.js + Express (Port 5000)
Routes Layer\nuserRoute / projectRoute\njobRoutes / contactRoutes
Middleware\nAuthMiddleware / validate.js\nuploadMiddleware / ErrorMiddleware
Controllers Layer\nuserController / jobController\nProjectReqestController
Services Layer\nuserService / jobService\nprojectRequestService
Repository Layer\nuserRepo / jobRepo\nprojectRepo / tokenRepo



Architecture Diagram

System Architecture

---

4. Project Structure

```
HouslyFinntechRealty/
├── client/                         # React Frontend
|   └── src/
|       ├── App.jsx                 # Route definitions
|       ├── main.jsx                # Entry point
|       ├── index.css               # Global styles
|       ├── pages/
|       |   ├── MainPage.jsx        # Main landing page wrapper
|       |   ├── SubMainPage.jsx     # Technology sub-page
|       |   └── Dashboard.jsx       # Admin dashboard (Protected)
```

```
|       ├── components/
|       |   ├── Navbar.jsx           # Navbar + Sidebar + Login Modal
|       |   ├── AdminNavbar.jsx      # Dashboard top navbar
|       |   ├── Home.jsx             # Hero + Features sections
|       |   ├── About.jsx            # About page
|       |   ├── Service.jsx          # Services page
|       |   ├── Contact.jsx          # Contact + Project Inquiry form
|       |   ├── Career.jsx           # Career opportunities overview
|       |   ├── Job.jsx              # Job listings with filter/sort
|       |   └── JobPost.jsx          # Job detail + Apply form
|       └── api/
|           ├── axiosInstance.js     # Axios + refresh token interceptor
|           ├── authApi.js           # login, logout, register, me
|           ├── dashboard.js         # Admin CRUD operations
|           ├── jobApplications.js   # Job application management
|           ├── jobpost.js           # Public job listings fetch
|           └── projectApi.js        # Project inquiry API
|
└── server/                          # Node.js Backend
    ├── server.js                    # Express app entry point
    ├── .env                         # Environment variables
    ├── config/
    |   └── db.js                    # MySQL connection pool
    ├── routes/
    |   ├── userRoute.js             # Auth routes
    |   ├── projectRoute.js          # Job posts + Project inquiry routes
    |   ├── jobRoutes.js             # Job application routes
    |   └── contactRoutes.js         # Contact form route
    ├── controller/
    |   ├── userController.js        # Auth controllers
    |   ├── jobController.js         # Application controllers
    |   ├── ProjectReqestController.js # Job post + inquiry controllers
```

```
    |   └── contactController.js     # Contact controller
    ├── service/
    |   ├── userService.js           # Auth business logic
    |   ├── jobService.js            # Application business logic
    |   └── projectRequestService.js # Job/inquiry business logic
    ├── repository/
    |   ├── userRepo.js              # User SQL queries
    |   ├── jobRepo.js               # Application SQL queries
    |   ├── projectRepo.js           # Job/inquiry SQL queries
    |   ├── tokenRepo.js             # Refresh token SQL queries
    |   └── contactRepo.js           # Contact SQL queries
    ├── middleware/
    |   ├── AuthMiddleware.js        # JWT cookie verification
    |   ├── validate.js              # express-validator rules
    |   ├── uploadMiddleware.js      # multer memory storage
    |   └── ErrorMiddleware.js       # Global error handler
    └── utility/
        └── token.js                 # JWT sign/verify helpers
```

---

5. Database Schema

Database Schema

5.1 Table: `users`

```sql
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
```

```
  name        VARCHAR(100)  NOT NULL,
  email       VARCHAR(150)  NOT NULL UNIQUE,
  password    VARCHAR(255)  NOT NULL,  -- bcrypt hash
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.2 Table: `refresh_tokens`

```
CREATE TABLE refresh_tokens (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  token       TEXT NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

5.3 Table: `job_applications`

```
CREATE TABLE job_applications (
  id                    INT AUTO_INCREMENT PRIMARY KEY,
  full_name             VARCHAR(100),
  email                 VARCHAR(150),
  phone                 VARCHAR(20),
  experience            VARCHAR(50),
  skills                TEXT,
  tenth_percentage      DECIMAL(5,2),
  twelfth_percentage    DECIMAL(5,2),
  graduation_percentage DECIMAL(5,2),
  college_name          VARCHAR(200),
```

```
  branch                VARCHAR(100),
  resume                LONGBLOB,         -- stored as binary
  resume_type           VARCHAR(50),      -- MIME type
  status                ENUM('pending','reviewed','selected','rejected') DEFAULT 'pending',
  created_at            TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.4 Table: `job_opportunities`

```
CREATE TABLE job_opportunities (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  title           VARCHAR(200),
  company         VARCHAR(200),
  type            VARCHAR(50),      -- Full-time, Part-time, Contract
  location        VARCHAR(100),
  locationType    VARCHAR(50),      -- Remote, Hybrid, On-site
  description     TEXT,
  postedDate      DATE,
  isNew           TINYINT(1) DEFAULT 0,
  salary          VARCHAR(100),
  aboutCompany    TEXT,
  responsibilities JSON,
  requirements     JSON,
  gains            JSON,
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.5 Table: `project_requests`

```sql
CREATE TABLE project_requests (
  id                INT AUTO_INCREMENT PRIMARY KEY,
  full_name         VARCHAR(100),
  company_name      VARCHAR(200),
  email             VARCHAR(150),
  phone             VARCHAR(20),
  inquiry_type      VARCHAR(100),
  service_required VARCHAR(200),
  message           TEXT,
  status            VARCHAR(50) DEFAULT 'pending',
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

---

6. API Reference

API Endpoints

6.1 Authentication APIs — `/api/auth`

Method	Endpoint	Auth Required	Description
`POST`	`/api/auth/register`	❌	Register new user
`POST`	`/api/auth/login`	❌	Login → sets httpOnly cookies
`POST`	`/api/auth/refresh`	❌ (cookie)	Refresh access token

Method	Endpoint	Auth Required	Description
`GET`	`/api/auth/me`	✅	Get logged-in user profile
`POST`	`/api/auth/logout`	✅	Logout current device
`POST`	`/api/auth/logout-all`	✅	Logout all devices
`GET`	`/api/auth/users`	✅	Get all users (admin)
`POST`	`/api/auth/users/create-developer`	✅	Create developer account
`DELETE`	`/api/auth/users/:id`	✅	Delete user

Login Request:

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "kamleshshah@gmail.com",
  "password": "Kamlesh@123"
}
```

Login Response:

```
HTTP 200 OK
Set-Cookie: accessToken=...; HttpOnly; SameSite=Strict; MaxAge=900
Set-Cookie: refreshToken=...; HttpOnly; SameSite=Strict; MaxAge=604800

{
```

```
  "success": true,
  "message": "Login successful",
  "user": {
    "id": 1,
    "name": "Kamlesh Shah",
    "email": "kamleshshah@gmail.com"
  }
}
```

6.2 Job Application APIs — `/api/jobs`

Method	Endpoint	Auth	Description
`POST`	`/api/jobs/apply`	❌	Submit job application with resume
`GET`	`/api/jobs/`	✅	Get all job applications
`GET`	`/api/jobs/resume/:id`	✅	Stream resume file
`PUT`	`/api/jobs/:id/status`	✅	Update application status
`DELETE`	`/api/jobs/:id`	✅	Delete application

Apply Request (multipart/form-data):

```
POST /api/jobs/apply
Content-Type: multipart/form-data

Fields: full_name, email, phone, experience, skills,
```

```
        tenth_percentage, twelfth_percentage, graduation_percentage,
        college_name, branch
File:   resume (PDF/DOC)
```

Update Status Request:

```
PUT /api/jobs/5/status
{ "status": "selected" }
```

6.3 Job Opportunities APIs — `/api/projects`

Method	Endpoint	Auth	Description
`GET`	`/api/projects/jobs`	❌	Get all job postings (public)
`POST`	`/api/projects/create-job`	✅	Create new job posting
`PUT`	`/api/projects/jobs/:id`	✅	Update job posting
`DELETE`	`/api/projects/jobs/:id`	✅	Delete job posting

Create Job Posting Body:

```
{
  "title": "Senior React Developer",
  "company": "Hously Finttech",
  "type": "Full-time",
  "location": "Ahmedabad",
```

```
  "locationType": "Hybrid",
  "description": "We are looking for...",
  "salary": "8-12 LPA",
  "aboutCompany": "Hously is a...",
  "responsibilities": ["Lead frontend development", "Code review"],
  "requirements": ["3+ years React", "TypeScript"],
  "gains": ["Health Insurance", "Flexible Hours"],
  "isNew": true
}
```

6.4 Project Request APIs — `/api/projects`

Method	Endpoint	Auth	Description
`POST`	`/api/projects/create`	❌	Submit project inquiry
`GET`	`/api/projects/get-all`	✅	Get all project inquiries
`PUT`	`/api/projects/:id/status`	✅	Update inquiry status
`DELETE`	`/api/projects/:id`	✅	Delete inquiry

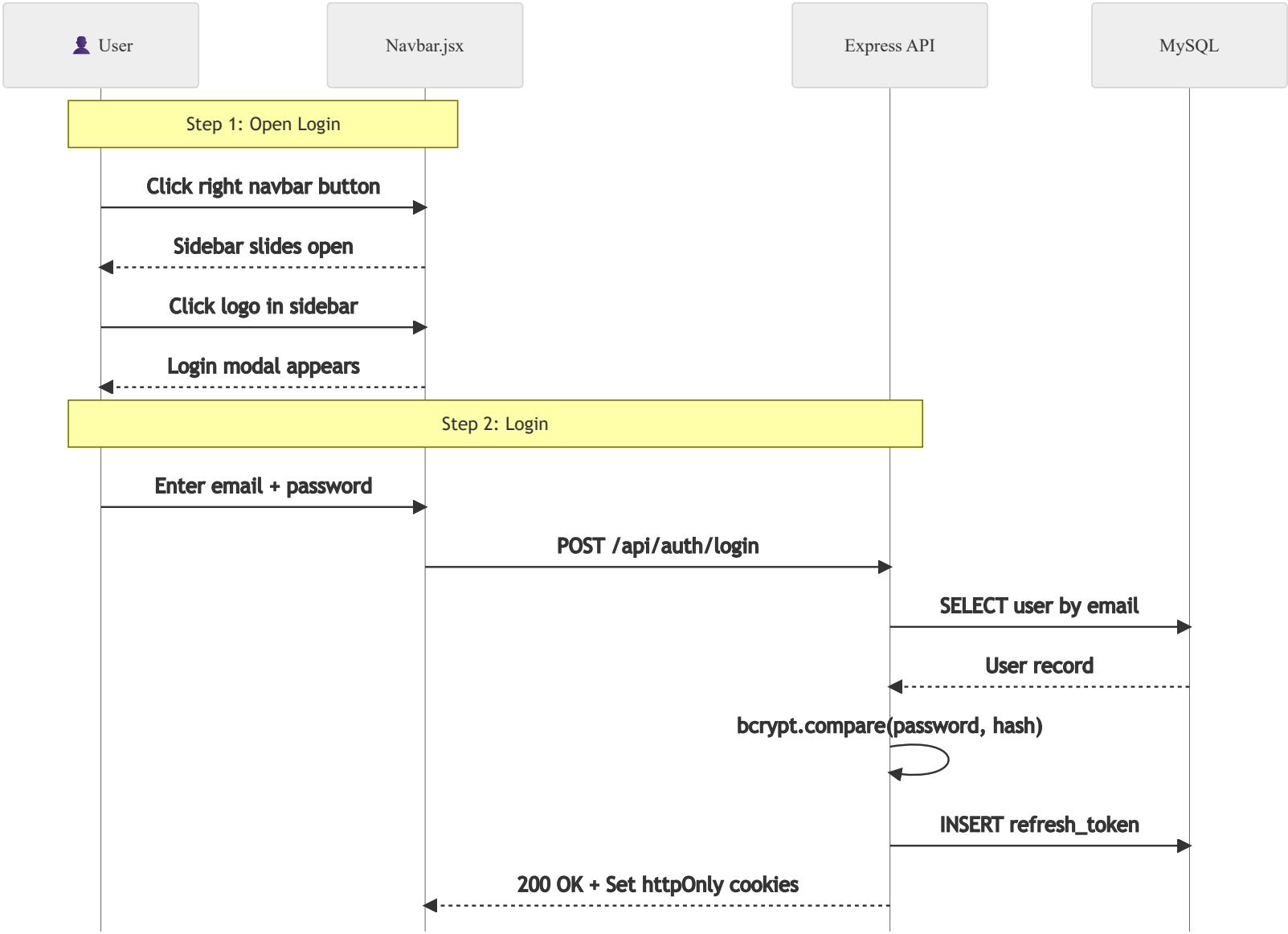
6.5 Contact API — `/api/contact`

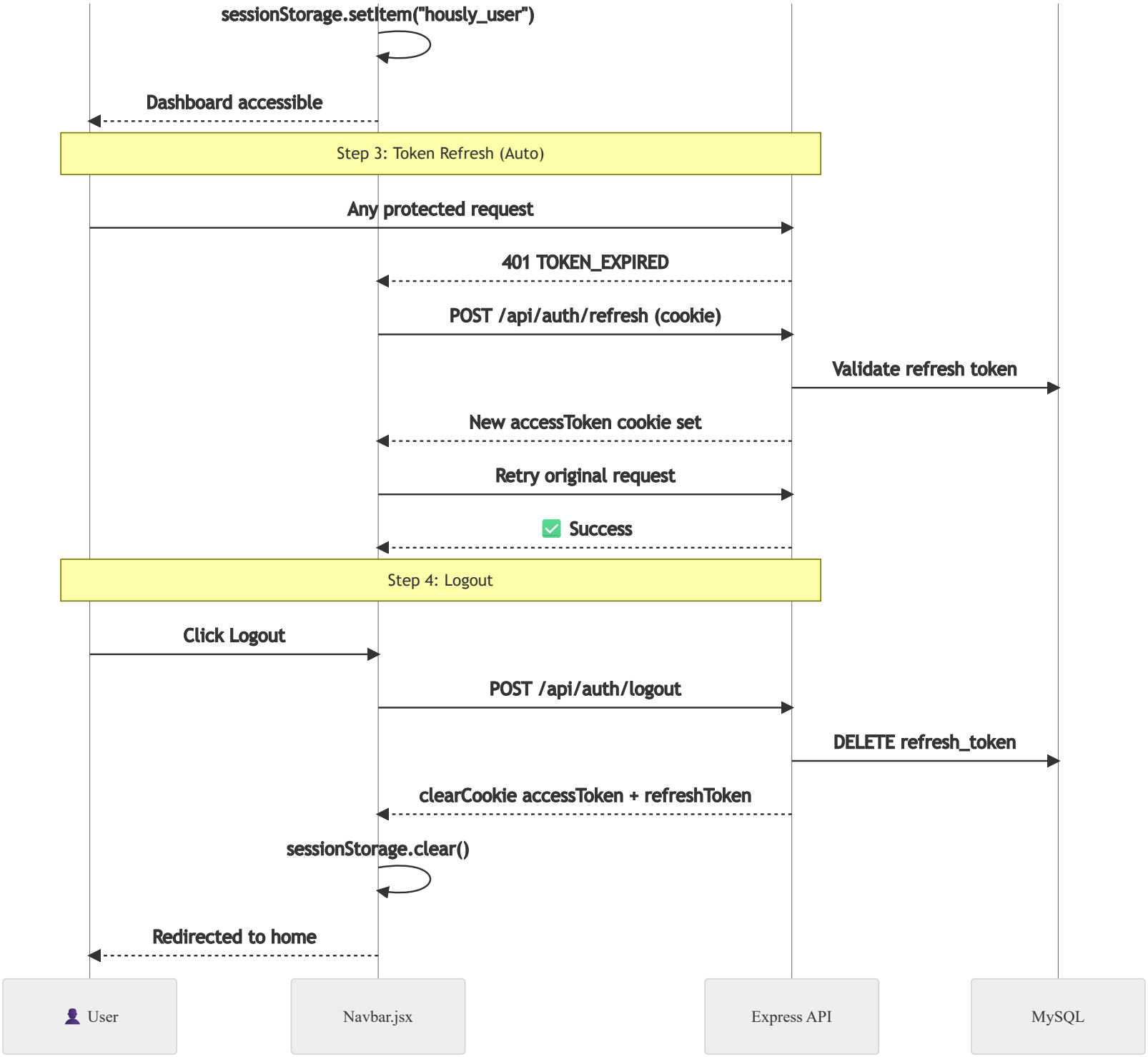
Method	Endpoint	Auth	Description
`POST`	`/api/contact`	❌	Submit contact form

---

7. Authentication Flow

Auth Flow


User
Navbar.jsx
Express API
MySQL
Step 1: Open Login
Click right navbar button
Sidebar slides open
Click logo in sidebar
Login modal appears
Step 2: Login
Enter email + password
POST /api/auth/login
SELECT user by email
User record
bcrypt.compare(password, hash)
INSERT refresh_token
200 OK + Set httpOnly cookies


sessionStorage.setItem("hously_user")
Dashboard accessible
Step 3: Token Refresh (Auto)
Any protected request
401 TOKEN_EXPIRED
POST /api/auth/refresh (cookie)
Validate refresh token
New accessToken cookie set
Retry original request
✅ Success
Step 4: Logout
Click Logout
POST /api/auth/logout
DELETE refresh_token
clearCookie accessToken + refreshToken
sessionStorage.clear()
Redirected to home
User
Navbar.jsx
Express API
MySQL

7.1 Token Strategy

Token	Storage	Expiry	Purpose
`accessToken`	httpOnly Cookie	**15 minutes**	Authenticate API requests
`refreshToken`	httpOnly Cookie	**7 days**	Silently renew access token
`user` object	sessionStorage	Session	UI state (name, email)

> [!IMPORTANT]
> Both tokens are stored as **httpOnly cookies** — they are **NOT accessible via JavaScript**, preventing XSS attacks. The Axios interceptor handles silent refresh automatically when accessToken expires.

Continued in Part 2: Frontend Architecture, Component Docs & Test Cases →